Secure Architecture Design: Smart Expense & Budget Management System

Project: Smart Expense & Budget Management System (Sec-D)

Basis: Security requirements SR01 to SR10 from the SRS

1. Overview

Purpose: Embed security principles into the architectural decisions of the Smart Expense & Budget Management System, so that design-level vulnerabilities are prevented before the system is built.

Scope: This document applies to the system's software components, data flows, deployment environments and integration points.

Key goals:

- Reduce the attack surface
- Ensure separation of concerns
- Support defense-in-depth
- Align with the threat model and the security requirements SR01 to SR10

All data in this application is personal financial data, and many users share the same system. The most important design rule is therefore that each user can reach only their own records.

2. Security Objective, Assets and Threats

Security objective: A user's financial data (income, expenses, budgets and savings goals) must stay confidential, must not be altered by anyone else, and must not be reusable through a stolen or old session.

Assets:

- Financial records: income, expenses, budgets and savings goals
- Credentials: passwords, which are stored only as hashes, and session tokens
- Reports exported from the system

Protection required from:

- Other users of the app, who must not read or change someone else's records
- Unauthenticated visitors and attackers, for example brute-force login attempts or injected input
- Anyone who gets hold of logs, error screens or a leaked database

3. The Asset in Each State

State	Where our data is	Controls
Origin point	The user types an income or expense into the React web form	Input validation (SR05)
In transit	Browser to the Express API to the PostgreSQL database	HTTPS/TLS (SR06), authenticated API (SR08)
At rest	PostgreSQL tables, including hashed passwords	Password hashing (SR02), per-user access (SR04)
In use	Dashboard charts, lists and exported reports shown to the logged-in user	Login required (SR01), session expiry (SR07)

4. Trust Boundaries

- **Untrusted:** the user's browser and everything it sends, including form input and URLs. Anyone can edit an ID in a URL, so the server never trusts it.
- **Trust boundary enforced by the Application Controller:** every request passes through it for a session check and input validation before it reaches any service.
- **Trusted (server side):** the business services and the PostgreSQL database, reachable only through the Application Controller.
- **Inside the boundary for a given record:** only the logged-in owner of that record. Other users and visitors are outside it.

5. Access Policy

A user can read and edit only their own income, expense, budget and savings data, and nothing else (SR04). Every record carries the owner's user ID, and every database query filters by the logged-in user's ID. This follows the principle of least privilege.

6. Security Requirements Mapped to Techniques

Requirement	What it says	Secure-architecture technique	Planned implementation
SR01	Authenticate before accessing financial data	Trust boundary, design protection	Login required; unauthenticated requests redirect to login
SR02	Passwords stored with secure hashing	Data at rest	Hash passwords with bcrypt or argon2
SR03	Secure tokens and proper session management	Design protection	Tokens that stop working after logout
SR04	Users access only their own data	Access policy, least privilege	Filter every query by user ID; test ID tampering early
SR05	Validate input before storing	Input validation layers	Server-side validation that rejects script tags, oversized text and non-numeric amounts
SR06	Sensitive data over HTTPS/TLS	Secure communication channels	No endpoint answers over plain HTTP
SR07	Sessions expire after inactivity	Defense-in-depth	Idle timeout, with the value to be decided (the SRS suggests 15 minutes)
SR08	Protect APIs from unauthorized access	Authenticated APIs	Reject calls without a valid token and log the attempt
SR09	Rate limit repeated login and API attempts	Rate-limited APIs	Block after a set number of failed tries; limit and lockout time to be fixed
SR10	No financial data or credentials in logs or errors	Reduce attack surface	Logs and error screens show no amounts, tokens or passwords

7. Architecture Review Checkpoints

1. **Are all sensitive components isolated?** The database is reachable only through the services, which are reachable only through the Application Controller.
2. **Are APIs authenticated, rate-limited and validated?** SR08, SR09 and SR05 cover this.
3. **Is encryption used where needed?** Transit is covered by SR06 and stored passwords by SR02. Whether the database itself is encrypted at rest is not specified in the SRS and should be decided.
4. **Are default configurations secure?** Secrets stay in environment variables and out of the repo, and error output hides internal details (SR10).

8. Architecture Threat Map

Component	High-value asset	Main threats	Requirements
React frontend	Session token, displayed data	Tampered input, script injection	SR05
Application Controller (API)	The single entry point for all requests	Unauthenticated access, brute force	SR01, SR08, SR09
Authentication and sessions	Credentials, tokens	Stolen or reused token, idle sessions left open	SR02, SR03, SR07
Business services	Calculations on user data	Reading another user's data	SR04
PostgreSQL database	Financial records, password hashes	Data leak, injection	SR02, SR04, SR05
Network	All traffic	Eavesdropping	SR06
Logs and error screens	Anything accidentally printed	Exposure of amounts, tokens, passwords	SR10

9. Outputs of the Architecture Phase

1. **Updated architecture diagram** with the trust boundary drawn around the Application Controller and services.

2. **Security architecture review report:** this document is the starting point, listing decisions, risks and mitigations.
3. **Change requests:** any gap found should be raised and recorded. Open items so far are the session timeout value, the rate-limit and lockout values, and encryption at rest.
4. **Architecture threat map:** the table in section 8.

Any place where the final build differs from the submitted SRS, SAD or test plan should also be written down, so it can be explained at the final demo.